Modern Topics in Computer Science (KHMT)

Assignment 1 — SAT Encodings versus Exact Solvers for the N-Queens Problem

–Manuscript Draft–

Manuscript Number:	Assignment 1 — Topic: N-Queens solver
Article Type:	Full length article
Keywords:	N-Queens , SAT encoding , at-most-one constraint , PySAT , CPLEX , Gurobi , CP-SAT
Corresponding Author:	Doan Trong Nghia VNU University of Engineering and Technology, Faculty of Information Technology
First Author:	Doan Trong Nghia
Order of Authors:	Doan Trong Nghia
Abstract:	The N-Queens problem asks for a placement of N non-attacking queens on an $N \times N$ chessboard. Although a solution exists for every $N \geq 4$, the problem is a standard benchmark for constraint solvers because its Boolean encoding grows quickly with N. This report compares seven ways of solving it: three SAT encodings of the at-most-one (AMO) constraint—Binomial (pairwise), Binary (bitwise) and Sequential counter—solved with the Glucose3 solver through the PySAT library, and four exact solvers: IBM CPLEX CP Optimizer, IBM CPLEX MIP, Gurobi MIP and Google OR-Tools CP-SAT. We analyse the size of each CNF encoding theoretically and by measurement, and benchmark all methods on 12 board sizes from $N = 32$ to $N = 4096$ under a time limit of 120 s, with five repetitions per configuration (medians are reported), the time split into import, model construction and solving, the peak memory, and both one thread and automatic thread settings. All returned boards were checked by an independent validator. The Binomial encoding is the fastest method only at $N = 32$ (0.053 s) and does not finish $N = 256$ because its cubic clause generation exceeds the limit; the Binary encoding is the best SAT encoding and the only one that solves $N = 512$ (28.0 s, of which 23.2 s is CNF generation in Python); the Sequential counter, although it has the fewest clauses, is the slowest, needing about ten times more conflicts than the Binary encoding at $N = 64$ and failing from $N = 100$. Among the exact solvers Gurobi MIP is the fastest for $N = 64$–256, whereas CPLEX CP Optimizer is the best from $N = 500$ and the only method that solves $N = 1000$, 2048 and 4096 (2.6 s at $N = 4096$ with one thread). More threads do not help, except for CP-SAT (about twice as fast at $N = 200$), and make CPLEX CP 11 times slower at $N = 4096$. Source code and raw results are publicly available.

SAT Encodings versus Exact Solvers for the N-Queens Problem

Doan Trong Nghia[a,*]

[a] *Faculty of Information Technology, VNU University of Engineering and Technology, Hanoi, Vietnam*

ARTICLE INFO

Keywords:
N-Queens
SAT encoding
at-most-one constraint
PySAT
CPLEX
Gurobi
CP-SAT

ABSTRACT

The N-Queens problem asks for a placement of N non-attacking queens on an $N \times N$ chessboard. Although a solution exists for every $N \geq 4$, the problem is a standard benchmark for constraint solvers because its Boolean encoding grows quickly with N. This report compares seven ways of solving it: three SAT encodings of the at-most-one (AMO) constraint—Binomial (pairwise), Binary (bitwise) and Sequential counter—solved with the Glucose3 solver through the PySAT library, and four exact solvers: IBM CPLEX CP Optimizer, IBM CPLEX MIP, Gurobi MIP and Google OR-Tools CP-SAT. We analyse the size of each CNF encoding both theoretically and by measurement ($N \leq 128$), and then benchmark all methods for $N \in \{32, 64, \ldots, 1024\}$ under a time limit of 60 s per run (one run per configuration). All returned boards were checked by an independent validator. The Binomial encoding is the fastest SAT encoding only for small boards and is limited by its cubic clause growth; among the configurations that were run, the Binary encoding is the only SAT encoding that solves $N = 512$ within the limit (20.9 s), while the Binomial and Sequential encodings were not run at this size; Gurobi and CPLEX CP solve every tested size up to $N = 1024$ (14.6 s and 2.8 s respectively), and are the fastest methods for $N \geq 256$. Surprisingly, the Sequential counter, which produces the fewest clauses, is the slowest SAT encoding in our runs, for a reason we could not identify; Gurobi and CPLEX were run with their default multi-thread settings, CP-SAT and the SAT solver with one thread. Source code and raw results are publicly available.

1. Introduction

The N-Queens problem is a classical combinatorial puzzle: place N queens on an $N \times N$ board so that no two queens share a row, a column or a diagonal. It dates back to the nineteenth century and is still used to test search algorithms and modelling techniques [1]. For $N \geq 4$ a solution always exists, so the interesting question is not satisfiability but *how efficiently a given formulation can be solved when N becomes large.*

Modern SAT solvers are highly effective on many structured problems, but they only accept formulas in conjunctive normal form (CNF). Hence a problem such as N-Queens must first be *encoded*. The part that dominates the encoding is the family of at-most-one (AMO) constraints: N rows, N columns and $2(2N - 1)$ diagonals each admit at most one queen. Many AMO encodings exist and they trade clause count against the number of auxiliary variables and the strength of unit propagation [2]. Alternatively, the problem can be stated directly in a higher-level language and given to an integer-programming (MIP) or constraint-programming (CP) solver.

This report studies the following question: *for which board sizes is a SAT encoding competitive with exact commercial or open-source solvers, and which AMO encoding scales best?* The contributions are:

- implementations of three SAT encodings (Binomial, Binary, Sequential counter) and four exact-solver models (CPLEX CP, CPLEX MIP, Gurobi MIP, OR-Tools CP-SAT) in Python;
- a theoretical and empirical comparison of the encoding sizes;
- a benchmark on boards up to 4096×4096 with five repetitions per configuration, solution validation, a separation of build and solve time, memory measurement and a comparison of thread settings, and a discussion of the observed behaviour and its limitations.

*Corresponding author
24021583@vnu.edu.vn (D.T. Nghia)

The rest of the report is organised as follows. Section 2 reviews related work. Section 3 defines the problem and the notation. Section 4 describes the SAT encodings and the exact models. Section 5 presents the experimental setup and results, and Section 6 concludes.

2. Related work

The N-Queens problem and its variants have a long history; a survey of known results is given by Bell and Stevens [1]. A solution for every $N \geq 4$ can be written down in linear time by closed-form constructions [8, 9], and min-conflicts local search finds solutions for very large boards in a few steps [10, 11]. Hence an exact solver is not needed in order to obtain *a* solution; the problem is used as a benchmark for modelling and solving techniques, and its generalisations are hard: completing a partially filled board is NP-complete [12].

For SAT-based solving, the choice of the encoding of the at-most-one constraint is the main design decision. The pairwise (binomial), bitwise (binary), sequential-counter [3], commander [4] and product [5] encodings trade the number of clauses against the number of auxiliary variables, and their relative speed depends on the solver and the instance [2, 13]. We use PySAT [7] to access the Glucose3 solver [6]. As exact alternatives we use the MIP solvers Gurobi [15] and CPLEX [16], the constraint programming solver CP Optimizer [14] (a CPLEX component) and the CP-SAT solver of Google OR-Tools [17].

3. Problem definition

Let $[N] = \{1, \ldots, N\}$. A board position is a pair $(i, j) \in [N]^2$ (i = row, j = column). Two distinct positions (i, j) and (i', j') *attack* each other iff

$$i = i' \quad \vee \quad j = j' \quad \vee \quad i - j = i' - j' \quad \vee \quad i + j = i' + j'.$$

A solution is a set of N positions, no two of which attack each other. Equivalently, it is a permutation π of $[N]$ ($\pi(i)$ = column of the queen in row i) such that the values $\pi(i) + i$ are pairwise distinct and the values $\pi(i) - i$ are pairwise distinct. This permutation view is used by the CP models in Section 4.5.

4. Methods

4.1. Base SAT model

We introduce a Boolean variable $x_{i,j}$ that is true iff a queen is placed at (i, j) (so there are N^2 primary variables, numbered $(i-1)N + j$ in the code). The constraints are

$$\bigvee_{j=1}^{N} x_{i,j} \quad \forall i \in [N] \qquad \text{(at least one queen per row)} \tag{1}$$

$$\bigvee_{i=1}^{N} x_{i,j} \quad \forall j \in [N] \qquad \text{(at least one queen per column)} \tag{2}$$

$$\mathrm{AMO}(x_{i,1}, \ldots, x_{i,N}) \quad \forall i \in [N] \qquad \text{(row)} \tag{3}$$

$$\mathrm{AMO}(x_{1,j}, \ldots, x_{N,j}) \quad \forall j \in [N] \qquad \text{(column)} \tag{4}$$

$$\mathrm{AMO}(\{x_{i,j} : i - j = d\}) \quad \forall d \in [-N+1, N-1] \qquad \text{(diagonals)} \tag{5}$$

$$\mathrm{AMO}(\{x_{i,j} : i + j = s\}) \quad \forall s \in [2, 2N] \qquad \text{(anti-diagonals)} \tag{6}$$

Constraints (1)–(4) together say *exactly one* queen per row and column; the diagonals only need "at most one", because there are $2N - 1$ of them and only N queens. Diagonals containing a single cell need no constraint. All three SAT variants share this model and differ only in how $\mathrm{AMO}(y_1, \ldots, y_m)$ is translated into clauses.

4.2. AMO encodings

Binomial (pairwise) encoding. The direct encoding forbids every pair:

$$\bigwedge_{1 \leq a < b \leq m} (\neg y_a \vee \neg y_b). \tag{7}$$

It needs no auxiliary variable and $\binom{m}{2}$ binary clauses, which propagate very well. In `binomial.py` the diagonal constraints are produced pair by pair from the board geometry. Since the row, column and diagonal groups have $\Theta(N)$ cells, the total number of clauses is $\Theta(N^3)$.

Table 1
Number of Boolean variables / clauses of the three SAT encodings (measured, before solving).

	Binomial		Binary		Sequential	
N	vars	clauses	vars	clauses	vars	clauses
8	64	744	174	728	270	600
16	256	6 352	572	3 928	1 182	2 728
32	1 024	52 640	1 850	19 800	4 926	11 592
64	4 096	428 864	6 136	95 576	20 094	47 752
128	16 384	3 462 784	21 238	447 832	81 150	193 800

Binary (bitwise) encoding. Introduce $\lceil \log_2 m \rceil$ fresh variables $b_1, \dots, b_k$ and give the a-th variable y_a the binary code of a. For every a and every bit t we add the clause $(\neg y_a \vee \ell_{a,t})$ where $\ell_{a,t} = b_t$ if bit t of a is 1 and $\neg b_t$ otherwise. If two variables $y_a, y_{a'}$ were both true, some bit t would force b_t and $\neg b_t$ simultaneously, a contradiction. The encoding uses $m\lceil \log_2 m \rceil$ clauses and $\lceil \log_2 m \rceil$ auxiliary variables per constraint, hence $\Theta(N^2 \log N)$ clauses in total. Each constraint receives its own fresh auxiliary variables (see `binary.py`).

Sequential counter encoding. Sinz's sequential counter [3] introduces $m-1$ auxiliary variables $s_1, \dots, s_{m-1}$, where s_a means "some variable among $y_1, \dots, y_a$ is true":

$$(\neg y_1 \vee s_1), \tag{8}$$

$$(\neg y_a \vee s_a),\ (\neg s_{a-1} \vee s_a),\ (\neg y_a \vee \neg s_{a-1}) \qquad 1 < a < m, \tag{9}$$

$$(\neg y_m \vee \neg s_{m-1}). \tag{10}$$

This gives $3m - 4$ clauses and $m - 1$ auxiliary variables for each constraint ($m \geq 3$; $m = 2$ is handled by one binary clause), so the total size is $\Theta(N^2)$.

Other encodings. The commander [4] and product [5] encodings (also available in PySAT) are natural candidates and are listed as future work (Section 6).

4.3. Size of the encodings

Table 1 reports the number of variables and clauses obtained by executing `generate_clauses` of each implementation, and Figure 1 plots the clause counts. The measured values agree with the asymptotic analysis: the Binomial encoding grows roughly eightfold when N doubles (cubic), the Binary one by about 4.7–5.4× ($N^2 \log N$) and the Sequential one by about 4× (quadratic). At $N = 128$ the pairwise encoding contains 3.46 million clauses, 17.9 times as many as the Sequential counter.

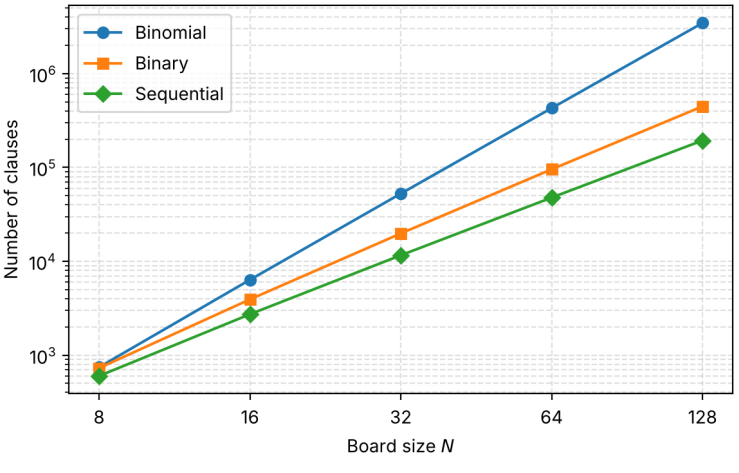


Figure 1: Number of clauses as a function of the board size N (log-log scale).

4.4. Solving the SAT instances

Each CNF is passed to the Glucose3 solver [6] through the PySAT toolkit [7]. A satisfying assignment is read from the model and converted into a 0/1 board. PySAT also offers CaDiCaL and MiniSat variants; only Glucose3 is used in this report in order to isolate the effect of the encoding.

4.5. Exact solvers

CPLEX CP Optimizer and OR-Tools CP-SAT. Both use the permutation model of Section 3: integer variables $q_i \in \{0, \dots, N-1\}$ (column of the queen in row i) and three global constraints

$$\text{AllDiff}(q_i), \qquad \text{AllDiff}(q_i + i), \qquad \text{AllDiff}(q_i - i). \tag{11}$$

The first constraint covers columns, the other two cover the diagonals; the row constraint is implicit. All exact solvers were run with one thread and with the automatic thread setting (Section 5). (`cplex_cp.py`, `cpsat.py`.)

CPLEX MIP and Gurobi MIP. Both use the same 0–1 program with binary variables $x_{i,j}$ and no objective (pure feasibility):

$$\sum_j x_{i,j} = 1 \qquad \forall i, \qquad\qquad \sum_i x_{i,j} = 1 \qquad \forall j, \tag{12}$$

$$\sum_{i-j=d} x_{i,j} \le 1 \qquad \forall d, \qquad\qquad \sum_{i+j=s} x_{i,j} \le 1 \qquad \forall s. \tag{13}$$

(`cplex_mip.py`, `gurobi_mip.py`.) This formulation has N^2 binaries and $6N - 2$ linear constraints, so its size is $\Theta(N^2)$ and independent of any AMO encoding.

5. Computational experiments

5.1. Experimental setup

Implementation and environment. All code is written in Python 3.12 under Windows 11, with `python-sat` (Glucose3), `docplex` with IBM ILOG CPLEX Studio (CP Optimizer and MIP), `gurobipy` and `ortools`. Machine: Intel Core i5-12450H (12th generation), 8 cores (12 logical processors), 8 GB RAM (7.6 GB usable). Software versions: PySAT (`python-sat`) 1.9.dev15 with the Glucose3 solver; Gurobi (`gurobipy`) 13.0.3; IBM ILOG CPLEX Optimization Studio 22.2.0 with the Python packages `cplex` 22.2.0.1 and `docplex` 2.32.264; Google OR-Tools 9.15.6755; `psutil` 5.9.8 for memory measurement.

Protocol. The driver `benchmark_v2.py` runs every run in a *separate process* and enforces a wall-clock limit of 120 s per run, so that a hung solver can be terminated; the solvers also receive the remaining time as their own time limit. Each configuration (method, N, thread setting) is run **five times** and the median, mean, standard deviation, minimum and maximum are stored. No (method, N) pair is skipped. If the first run of a configuration reaches the time limit, the configuration is recorded as a time-out (TO) and is not repeated, since further runs would only waste time. Every run is timed in three parts: *import* of the solver modules, *build* (model or CNF generation, including loading the clauses into the solver) and *solve*; the total time is their sum. The peak resident memory of the worker process and its children is sampled with `psutil` (a sampled value is a lower bound of the true peak). For the SAT solver the numbers of conflicts, decisions and propagations are recorded (PySAT `accum_stats`). Every returned board is verified by an independent checker (one queen per row, no two queens in the same column or diagonal); the column *Valid* of the result file is true for all finished runs (all 476 runs of the benchmark are included in the raw data).

Thread settings. Gurobi, CPLEX MIP, CPLEX CP Optimizer and OR-Tools CP-SAT were each run twice: with *one* thread and with the solver's automatic setting (parameter value 0, which uses the 12 logical processors). Glucose3 is sequential and is run once. Unless stated otherwise, the main comparison (Table 2 and Figure 2) uses *one thread for every method*, so that all methods have the same computational resources; the effect of the thread setting is analysed separately (Table 5).

Instances. $N \in \{32, 64, 100, 128, 200, 256, 500, 512, 1000, 1024, 2048, 4096\}$, i.e. powers of two and also sizes that are not powers of two. The solvers are called with default parameters except for the time limit and the thread count.

Table 2
Total time in seconds, median ± standard deviation of five runs (one thread). TO: no solution within 120 s in the first run. The best median in each row is in bold.

	SAT (Glucose3)			Exact solvers			
N	Binomial	Binary	Sequential	CPLEX CP	CPLEX MIP	Gurobi	CP-SAT
32	**0.053**±0.002	0.241±0.012	0.135±0.004	0.755±0.101	0.795±0.082	0.089±0.290	0.732±0.146
64	0.372±0.015	0.191±0.035	2.97±0.590	0.756±0.037	0.893±0.017	**0.078**±0.005	0.917±0.017
100	1.43±0.026	0.331±0.013	TO	0.773±0.033	1.08±0.013	**0.108**±0.006	1.42±0.021
128	3.04±0.182	0.594±0.007	TO	0.774±0.094	1.41±0.094	**0.167**±0.343	2.76±0.291
200	31.8±5.25	1.53±0.068	TO	0.798±0.089	2.57±0.058	**0.304**±0.352	56.6±3.02
256	TO	2.48±0.046	TO	0.851±0.148	7.47±0.112	**0.449**±0.408	42.6±1.18
500	TO	24.7±0.580	TO	**0.886**±0.111	70.9±1.60	1.77±0.292	TO
512	TO	28.0±1.03	TO	**0.936**±0.496	81.4±2.37	1.82±0.319	TO
1000	TO	TO	TO	**0.997**±0.468	TO	TO	TO
1024	TO	TO	TO	**1.01**±0.771	TO	17.6±1.02	TO
2048	TO	TO	TO	**1.63**±0.823	TO	TO	TO
4096	TO	TO	TO	**2.61**±0.609	TO	TO	TO

Table 3
Median build time / solve time in seconds (peak memory in MB in parentheses), one thread. TO: time-out.

	$N = 64$	$N = 128$	$N = 256$	$N = 512$	$N = 1024$
Binomial	0.348 / 0.007 (96)	2.93 / 0.088 (685)	TO	TO	TO
Binary	0.096 / 0.075 (34)	0.459 / 0.119 (97)	2.29 / 0.176 (386)	23.2 / 4.37 (1722)	TO
Sequential	0.025 / 2.93 (53)	TO	TO	TO	TO
CPLEX CP	0.001 / 0.003 (74)	0.001 / 0.006 (74)	0.002 / 0.008 (76)	0.004 / 0.038 (82)	0.007 / 0.150 (109)
CPLEX MIP	0.098 / 0.016 (84)	0.537 / 0.094 (100)	5.15 / 1.39 (186)	72.4 / 8.09 (501)	TO
Gurobi	0.029 / 0.005 (27)	0.117 / <0.001 (33)	0.387 / 0.016 (70)	1.68 / 0.087 (196)	16.5 / 1.04 (704)
CP-SAT	0.001 / 0.274 (83)	0.002 / 2.04 (107)	0.003 / 41.7 (260)	TO	TO

5.2. Results

Table 2 shows the total time (median ± standard deviation of five runs, one thread) and Figure 2 the medians on a logarithmic scale. TO means that the first run did not return a solution within 120 s. For a few time-outs the process was terminated by the driver before the solver had started (for example during CNF generation), so no build/solve split exists for them.

5.3. Discussion

Where the time goes. Table 3 separates model construction from solving. For the SAT encodings and for the MIP solvers the *build* phase, which is executed in pure Python, dominates: for the Binomial encoding at $N = 128$ the CNF generation takes 2.93 s and Glucose3 only 0.09 s; for the Binary encoding at $N = 512$ it takes 23.2 s against 4.4 s of solving; for CPLEX MIP at $N = 512$ the model construction takes 72.4 s against 8.1 s of solving, and for Gurobi at $N = 1024$ it takes 16.5 s against 1.0 s. The Binomial encoding at $N = 256$ did not finish because the clause generation alone exceeded the limit (the recorded build time was 123.7 s, so the SAT solver was never started); at $N = 200$ the 13.3 million clauses already need 2.6 GB of memory. Hence the ranking of these methods is partly a statement about the Python modelling layer and not only about the underlying solver. The floor of 0.7–0.8 s of CPLEX CP, CPLEX MIP and CP-SAT at small N is almost entirely the one-off import of the Python package (median import time 0.80 s, 0.81 s and 0.69 s; 0.04 s for Gurobi and 0.02 s for the SAT modules). Without it, CPLEX CP needs only 2–8 ms to solve boards up to $N = 256$ and 0.15 s at $N = 1024$, which shows that its search effort is almost negligible on this problem.

SAT encodings. For the smallest board ($N = 32$) the pairwise encoding is the fastest of all seven methods (0.053 s), because its binary clauses are propagated efficiently and the formula is still small. Its cubic growth, however, makes it unusable soon after: it solves $N = 200$ in 31.8 s (29.8 s of which is CNF generation) and does not finish $N = 256$. The Binary encoding is slower than the pairwise one at $N = 32$ (0.24 s) but faster from $N = 64$ on, and it is the only SAT encoding that solves $N = 500$ and $N = 512$ (24.7 s and 28.0 s);

Table 4
Glucose3 statistics (median of five runs; the conflict and decision counts were identical in all runs): clauses, conflicts, decisions, solving time and solving time per conflict.

N	Encoding	Clauses	Conflicts	Decisions	Solve (s)	μs/conflict
32	Binomial	52 640	166	903	0.002	12
32	Binary	19 800	6 507	8 715	0.212	33
32	Sequential	11 592	2 736	4 593	0.113	41
64	Binomial	428 864	245	3 224	0.007	27
64	Binary	95 576	1 693	9 932	0.075	44
64	Sequential	47 752	17 885	23 432	2.93	164
100	Binomial	1 646 900	351	10 823	0.019	55
100	Binary	269 024	478	7 453	0.040	84
100	Sequential			TO (>120 s)		
128	Binomial	3 462 784	374	15 815	0.088	236
128	Binary	447 832	974	18 266	0.119	122
128	Sequential			TO (>120 s)		
200	Binomial	13 253 800	3 184	80 342	1.67	524
200	Binary	1 236 200	1 169	38 913	0.189	161
200	Sequential			TO (>120 s)		

it fails at $N \geq 1000$, where we only know that no solution was produced within 120 s (extrapolating the quasi-quadratic growth of its CNF generation, 23 s at $N = 512$, suggests that clause generation already uses most of the limit). The Sequential counter has the *fewest* clauses (Table 1) yet is the slowest SAT encoding: 2.97 s at $N = 64$ against 0.19 s (Binary) and 0.37 s (Binomial), and no solution within 120 s for $N \geq 100$. Table 4 gives a data-based explanation. At $N = 64$ Glucose3 needs 17 885 conflicts on the Sequential encoding, 10.6 times as many as on the Binary encoding (1 693) and 73 times as many as on the Binomial one (245); in addition each conflict is more expensive on the Sequential formula (164 μs against 44 μs and 27 μs), which is consistent with its long implication chains and its larger number of auxiliary variables (20 094 against 4 096 primary variables). Between $N = 32$ and $N = 64$ the conflict count of the Sequential encoding grows by 6.5 times, whereas it *decreases* for the Binary encoding. We did not test why the auxiliary-variable structure leads to more conflicts (for example through the VSIDS scores or the initial phase), so this remains a hypothesis; the measurement shows that the clause count is not a good predictor of solving time, and that solving effort is not monotone in N for the Binary encoding. Glucose3 is deterministic: the five runs of a SAT configuration always had the same conflict count, so their spread reflects timing noise only, not search variability (random initial phases are available in the driver but were not used).

Exact solvers. CPLEX CP is by far the most scalable method: its time grows slowly from 0.76 s ($N = 64$) to 0.94 s ($N = 512$), 1.01 s ($N = 1024$), 1.63 s ($N = 2048$) and 2.61 s ($N = 4096$) with a single thread, and it is the only method that solves $N = 1000$, 2048 and 4096. Gurobi MIP is the fastest method for $N = 64$–256 (0.08–0.45 s), and CPLEX CP overtakes it from $N = 500$ (0.89 s against 1.77 s). Gurobi's behaviour at large N is instance dependent: it solves $N = 1024$ in 17.6 s (1.0 s of solving) but reached the time limit at $N = 1000$ after 18 s of model construction (the single run was not repeated and we have no explanation). CPLEX MIP solves up to $N = 512$ (81.4 s, mainly model construction) and times out at $N \geq 1000$. OR-Tools CP-SAT, which uses the same permutation model with three `AllDifferent` constraints as CPLEX CP, is much slower and irregular with one thread: 0.27 s of solving at $N = 64$, 2.0 s at $N = 128$, 55.9 s at $N = 200$, 41.7 s at $N = 256$ and time-outs from $N = 500$; the numbers of conflicts (26 255 at $N = 200$ against 4 831 at $N = 256$) show that $N = 200$ is simply a harder instance for its search than $N = 256$. This difference between two solvers on identical constraints comes from their propagation and search strategies; we did not investigate it further. The time of Gurobi and CPLEX CP shows a few large outliers in the first of the five runs (for example Gurobi at $N = 32$: 0.73 s in the first run against 0.07–0.11 s in the other four), a cold-start effect that inflates the standard deviations in Table 2; the medians are not affected.

Table 5
Median total time in seconds with one thread (1) and with the automatic thread setting, 12 logical processors (A). TO: time-out.

	CPLEX CP		CPLEX MIP		Gurobi		CP-SAT	
N	1	A	1	A	1	A	1	A
128	0.774	0.930	1.41	1.38	0.167	0.162	2.76	2.23
200	0.798	0.858	2.57	2.59	0.304	0.290	56.6	27.5
256	0.851	0.883	7.47	7.36	0.449	0.437	42.6	42.9
500	0.886	0.945	70.9	76.4	1.77	1.67	TO	TO
512	0.936	0.973	81.4	82.6	1.82	1.80	TO	92.2
1024	1.01	1.36	TO	TO	17.6	16.6	TO	TO
2048	1.63	3.56	TO	TO	TO	TO	TO	TO
4096	2.61	28.4	TO	TO	TO	TO	TO	TO

Effect of the thread setting. Table 5 shows that more threads do not help on this problem, with one exception. Gurobi and CPLEX MIP are practically unaffected (differences of at most 8%, within the noise). CPLEX CP becomes *slower* with the automatic setting, increasingly so for large boards: 3.56 s against 1.63 s at $N = 2048$ and 28.4 s against 2.6 s at $N = 4096$, where the peak memory grows from 0.4 GB to 3.8 GB. Only CP-SAT benefits from its parallel portfolio: at $N = 200$ it halves the time (27.5 s against 56.6 s), and it solves $N = 512$ (92.2 s) where the single-thread run times out, but $N = 500$ and $N \geq 1000$ still time out, and its peak memory rises to about 1.6 GB at $N = 256$. (The conflict counts that CP-SAT reports in multi-thread mode are not meaningful and are not analysed.) The ranking of the methods therefore does not depend on the thread setting.

SAT versus exact solvers. On very small boards the best SAT encoding is the fastest method ($N = 32$, Binomial); for $N = 64$–128 the Binary encoding (0.19–0.59 s) is faster than CPLEX CP, CPLEX MIP and CP-SAT and only Gurobi is faster; at $N = 100$ it is already 3.1 times slower than Gurobi (0.33 s against 0.11 s) and at $N = 200$ it is also slower than CPLEX CP. These comparisons at small N are strongly influenced by the start-up cost of the CPLEX and OR-Tools packages (see above). For $N \geq 500$, Gurobi and CPLEX CP are faster than every SAT encoding (at $N = 512$ the Binary encoding takes 28.0 s against 1.8 s for Gurobi and 0.94 s for CPLEX CP), and beyond $N = 1024$ only CPLEX CP finishes. Hence a SAT approach with a compact encoding is a reasonable choice for boards up to about $N = 128$, while for larger boards the constraint-programming or MIP formulation is clearly better. Dedicated algorithms, such as closed-form constructions or min-conflicts local search [8, 10], were not measured in this study.

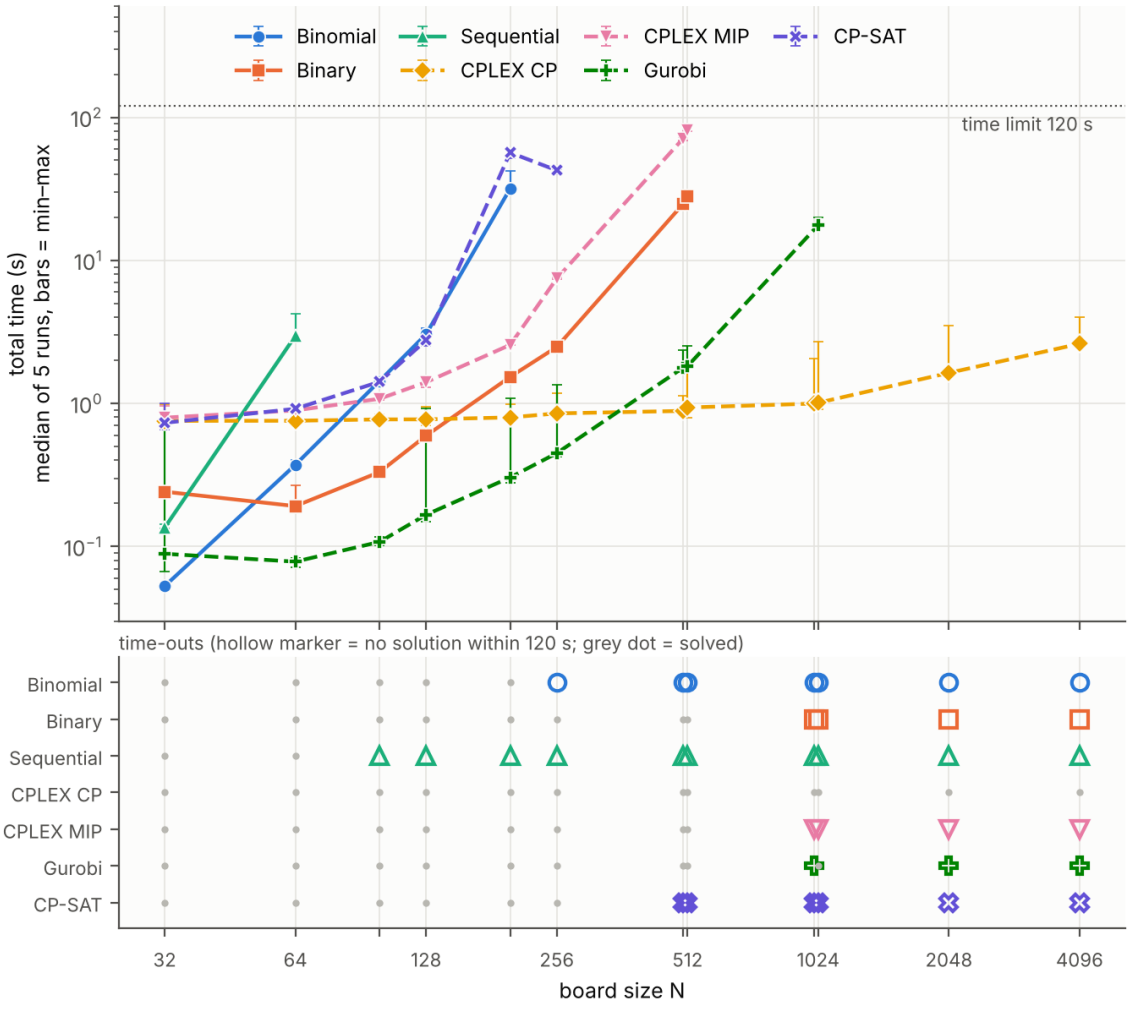


Figure 2: Total time versus board size for the seven methods with one thread (log–log scale; markers are medians of five runs, bars show the minimum and maximum). The lower panel marks the configurations that reached the 120 s limit.

5.4. Threats to validity

- Each configuration was run five times on one laptop. The SAT solver and CP-SAT with one thread are deterministic, so their repetitions only measure timing noise; random seeds or phases were not varied. The configurations that timed out in the first run were not repeated, so a time-out is a single observation. The first run of each configuration includes a cold-start effect (see above); medians were used for this reason.

- The measured time includes the Python overhead of import and model/CNF generation, which dominates for the SAT encodings and the MIP solvers (Table 3). The comparison between MIP, CP and SAT is therefore a comparison of our Python implementations and not of the solvers alone; a CNF generator written in a compiled language would favour the SAT encodings.

- Peak memory was sampled and is a lower bound. The machine has 7.6 GB of memory, which limits large instances (for example the Binomial encoding at $N = 200$ already uses 2.6 GB).

- Only the first solution is requested; default solver parameters were used, one SAT solver (Glucose3) and three AMO encodings were compared (commander and product encodings are not covered); the time limit is 120 s.

- The behaviour of Gurobi at $N = 1000$ against $N = 1024$ and of CP-SAT at $N = 200$ against $N = 256$ shows a strong dependence on the instance, but each was observed on a single size, so no general rule can be derived.
- An earlier single-run benchmark of the same code produced different absolute times for several cells (for example 6.5 s for CPLEX CP at $N = 512$, against a median of 0.94 s here), which we could not explain; for this reason only the repeated measurements are reported.

6. Conclusion

We implemented and compared three SAT encodings and four exact solvers for the N-Queens problem, using five repetitions, a 120 s limit, boards up to $N = 4096$ and two thread settings. The experiments show that (i) the pairwise encoding is best only for the smallest board and its $\Theta(N^3)$ clause growth stops it before $N = 256$; (ii) the Binary encoding is the best SAT encoding (solved up to $N = 512$), but most of its time is CNF generation in Python, not solving; (iii) the Sequential counter, although smallest in size, is the slowest because Glucose3 needs far more conflicts (10.6 times as many as for the Binary encoding at $N = 64$) and each conflict is more expensive, so the encoding size is not a sufficient criterion; (iv) Gurobi MIP is the fastest method for $N = 64$–256, while CPLEX CP Optimizer is the best from $N = 500$ and the only method that solves $N = 1000$, 2048 and 4096; and (v) additional threads do not improve the solvers on this problem, except for CP-SAT, and substantially degrade CPLEX CP at large N. A SAT approach with a compact encoding is thus competitive up to about $N = 128$; for larger boards a CP or MIP formulation is preferable.

Future work includes the commander and product encodings, additional PySAT solvers (for example CaDiCaL and MiniSat), randomised initial phases and seeds to measure search variability, a faster (compiled) CNF and model generator to separate solver performance from Python overhead, symmetry breaking and solution-structure hints, tuned CP-SAT parameters, and the comparison with dedicated algorithms such as closed-form constructions and local search.

Data availability

The source code of all solvers, the benchmark driver (`benchmark_v2.py`) and the raw result files (files named `benchmark_v2_{runs,summary,env}_20261015_131718`) are available at https://github.com/onlynghiax/KHMT.

Declaration of competing interest

The author declares that there is no known competing financial interest or personal relationship that could have influenced the work reported in this report.

References

[1] J. Bell and B. Stevens, A survey of known results and research areas for n-queens, *Discrete Mathematics* 309(1) (2009) 1–31.

[2] S. Prestwich, CNF encodings, in: A. Biere et al. (eds.), *Handbook of Satisfiability*, IOS Press, 2009, pp. 75–97.

[3] C. Sinz, Towards an optimal CNF encoding of Boolean cardinality constraints, in: *Principles and Practice of Constraint Programming (CP 2005)*, LNCS 3709, Springer, 2005, pp. 827–831.

[4] W. Klieber and G. Kwon, Efficient CNF encoding for selecting 1 from N objects, in: *Fourth Workshop on Constraints in Formal Verification (CFV)*, 2007.

[5] J. Chen, A new SAT encoding of the at-most-one constraint, in: *Proc. ModRef Workshop on Constraint Modelling and Reformulation*, 2010.

[6] G. Audemard and L. Simon, Predicting learnt clauses quality in modern SAT solvers, in: *Proc. IJCAI 2009*, 2009, pp. 399–404.

[7] A. Ignatiev, A. Morgado and J. Marques-Silva, PySAT: A Python toolkit for prototyping with SAT oracles, in: *SAT 2018*, LNCS 10929, Springer, 2018, pp. 428–437.

[8] E. J. Hoffman, J. C. Loessi and R. C. Moore, Constructions for the solution of the m queens problem, *Mathematics Magazine* 42(2) (1969) 66–72.

[9] B. Abramson and M. Yung, Divide and conquer under global constraints: a solution to the n-queens problem, *Journal of Parallel and Distributed Computing* 6(3) (1989) 649–662.

[10] S. Minton, M. D. Johnston, A. B. Philips and P. Laird, Minimizing conflicts: a heuristic repair method for constraint satisfaction and scheduling problems, *Artificial Intelligence* 58(1–3) (1992) 161–205.
[11] R. Sosič and J. Gu, Efficient local search with conflict minimization: a case study of the n-queens problem, *IEEE Transactions on Knowledge and Data Engineering* 6(5) (1994) 661–668.
[12] I. P. Gent, C. Jefferson and P. Nightingale, Complexity of n-Queens completion, *Journal of Artificial Intelligence Research* 59 (2017) 815–848.
[13] A. M. Frisch and P. A. Giannaros, SAT encodings of the at-most-k constraint: some old, some new, some fast, some slow, in: *Proc. ModRef Workshop on Constraint Modelling and Reformulation*, 2010.
[14] P. Laborie, J. Rogerie, P. Shaw and P. Vilím, IBM ILOG CP Optimizer for scheduling, *Constraints* 23(2) (2018) 210–250.
[15] Gurobi Optimization, LLC, Gurobi Optimizer Reference Manual, version 13.0, https://www.gurobi.com.
[16] IBM, IBM ILOG CPLEX Optimization Studio, version 22.2.0, https://www.ibm.com/products/ilog-cplex-optimization-studio.
[17] L. Perron and V. Furnon, Google OR-Tools, version 9.15, https://developers.google.com/optimization.